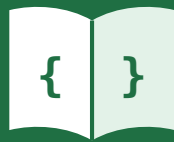

Module 05 — Spring JDBC and PostgreSQL

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Database Access Layers in Spring	2
2.2 Transactions	2
2.3 Development and Test Databases	3
3. Step-by-Step Examples	3
3.1 A Database with Docker Compose	3
3.2 Schema Management with Flyway	4
3.3 SQL with JdbcClient	5
3.4 An Aggregate with Spring Data JDBC	6
3.5 Transactions	7
3.6 @TransactionalEventListener	8
3.7 Testing with Testcontainers	9
4. Common Mistakes and Best Practices	10
5. Summary	11
6. Further Reading	11

1. Learning Goals

By the end of this module you will be able to:

- Start the development database automatically with Spring Boot's Docker Compose support
- Version the database schema with Flyway migrations
- Write SQL queries with `JdbcTemplate` and map the results to records
- Save and load an aggregate as a whole with Spring Data JDBC
- Write consistent operations with `@Transactional`, and know how rollback and propagation behave
- React only to committed work with `@TransactionalEventListener`
- Test against a real PostgreSQL with `Testcontainers`

Prerequisites: Modules 01-03, basic SQL · **Estimated time:** 5 hours · **Docker required**

2. Concepts

2.1 Database Access Layers in Spring

Layer	What do you write?	When?
<code>JdbcTemplate</code>	SQL	Full control over SQL, reports, bulk operations
Spring Data JDBC	Aggregate classes + a repository interface	Simple, predictable object-table mapping
Spring Data JPA (Module 06)	Entities + repositories	Complex relationships, lazy loading, caching

Underneath all of them sit a `DataSource` (the **HikariCP** connection pool by default) and the JDBC driver. Spring translates SQL errors into a database-independent exception hierarchy (e.g. `DataIntegrityViolationException`).

2.2 Transactions

Either **all** changes in a transaction become permanent (commit), or **none** do (rollback). `@Transactional` manages this through a proxy:

- A transaction starts when the method starts, and commits when it ends normally.

- A `RuntimeException` (or `Error`) causes a rollback. **Checked exceptions commit by default.**

Propagation	Meaning
REQUIRED (default)	Join the current transaction, or start a new one
REQUIRES_NEW	Suspend the current transaction, open and close one of its own
MANDATORY	A current transaction is required, otherwise fail
NOT_SUPPORTED	Run outside of any transaction

2.3 Development and Test Databases

- **Development:** the `spring-boot-docker-compose` dependency starts the PostgreSQL from the root `compose.yaml` during `spring-boot:run`. The connection details are read from the container, so no `spring.datasource.*` is needed.
- **Tests:** `Testcontainers` starts a disposable, real PostgreSQL for every test run, and `@ServiceConnection` hands its connection details to Boot. Testing against the real database instead of a stand-in such as H2 also catches database-specific behaviour (constraints, types, error codes).

3. Step-by-Step Examples

With Docker running, start the application:

```
./mvnw -pl modules/05-data-jdbc-postgres/lesson -am spring-boot:run
```

3.1 A Database with Docker Compose

Goal: work with PostgreSQL through a single command, without starting containers by hand.

```
docker:
  compose:
    # Reuse the root compose.yaml; only the listed profiles are started.
    file: ../../../../compose.yaml
    profiles:
```

```

    active: postgres
    # Keep containers running between restarts (stop them with: docker compose --profile
↪ all down)
    lifecycle-management: start-only

```

Expected output:

```

... DockerCli : Container spring-boot-edu-postgres-1 Started
... DockerCli : Container spring-boot-edu-postgres-1 Healthy
== 3.1 DataSource (from Docker Compose)
jdbc:postgresql://127.0.0.1:5432/bookstore?ApplicationName=05-data-jdbc-postgres ·
↪ PostgreSQL 18.6 ... · pool HikariDataSource

```

i Note

In the root `compose.yaml`, the PostgreSQL service uses the `pgvector/pgvector` image. Boot recognises services by image name, and this image is not called “postgres”. That is why the service carries the label `org.springframework.boot.service-connection: postgres`.

3.2 Schema Management with Flyway

Goal: version the schema together with the code and apply it in the same order everywhere.

The `V<version>__<description>.sql` files in `src/main/resources/db/migration/` run in order when the application starts. Applied migrations are recorded in the `flyway_schema_history` table:

```

create table author (
    id    bigserial primary key,
    name text      not null
);

create table book (
    id          bigserial primary key,
    isbn         varchar(13) not null unique,
    title       text       not null,
    author_id   bigint      references author (id),
    price       numeric(10,2) not null check (price >= 0),
    stock       integer      not null default 0 check (stock >= 0) -- the database guards
↪ the rule too
);

```

Expected output:

```
... DbMigrate : Migrating schema "public" to version "1 - create schema"
... DbMigrate : Migrating schema "public" to version "2 - seed books"
== 3.2 Flyway
V1 create schema
V2 seed books
```

Its test: FlywayMigrationTest

!! Caution

Never change an applied migration file. Flyway notices the checksum difference and refuses to start the application. Add a new version (V3__...) instead.

3.3 SQL with JdbcTemplate

Goal: write the SQL yourself, and leave connections, parameters and mapping to Spring.

```
public List<Book> findAll() {
    return jdbc.sql("select id, isbn, title, price, stock from book order by id")
        .query(Book.class) // columns → record components
        .list();
}

public Optional<Book> findByIsbn(String isbn) {
    return jdbc.sql("select id, isbn, title, price, stock from book where isbn = :isbn")
        .param("isbn", isbn) // named parameter: no SQL injection
        .query(Book.class)
        .optional();
}
```

The result of a join can be mapped to a record defined just for that query. The `author_name` column goes to the `authorName` component:

```
public List<BookWithAuthor> findWithAuthors() {
    return jdbc.sql("""
        select b.title, a.name as author_name
        from book b join author a on a.id = b.author_id
        order by b.title""")
        .query(BookWithAuthor.class)
        .list();
}
```

A `KeyHolder` fetches the key generated by the database:

```

public long insert(Book book) {
    KeyHolder keys = new GeneratedKeyHolder();
    jdbc.sql("insert into book (isbn, title, price, stock) values (:isbn, :title, :price,
    ↪ :stock)")
        .paramSource(book) // parameters from the record's
        ↪ components
        .update(keys, "id"); // ask PostgreSQL for the
        ↪ generated id
    return keys.getKeyAs(Long.class);
}

```

Expected output:

```

== 3.3 JdbcClient
Book[id=1, isbn=9780134685991, title=Effective Java, price=89.90, stock=5]
...
BookWithAuthor[title=Effective Java, authorName=Joshua Bloch]

```

Its test: book/JdbcBookRepositoryTest. Negative stock hits the database's check constraint and arrives as a DataIntegrityViolationException.

3.4 An Aggregate with Spring Data JDBC

Goal: store objects that change together (an order and its lines) as one unit.

The aggregate root and its child collection:

```

@Table("purchase_order") // "order" is a reserved word
↪ in SQL
public record PurchaseOrder(
    @Id @Nullable Long id, // null → INSERT, otherwise UPDATE
    String customerEmail,
    Instant createdAt,
    @MappedCollection(idColumn = "purchase_order") Set<OrderLine> lines) {

    public static PurchaseOrder create(String customerEmail, Set<OrderLine> lines) {
        return new PurchaseOrder(null, customerEmail, Instant.now(), lines);
    }

    public PurchaseOrder withId(Long newId) { // Spring Data uses this to set
        ↪ the generated id
        return new PurchaseOrder(newId, customerEmail, createdAt, lines);
    }
}

```

You write one repository per aggregate root, and Spring Data generates the implementation:

```

public interface PurchaseOrderRepository extends ListCrudRepository<PurchaseOrder, Long>
↳ {

    List<PurchaseOrder> findByCustomerEmail(String customerEmail);    // derived from
↳ the method name

    @Query("""
        select po.id from purchase_order po
        join order_line ol on ol.purchase_order = po.id
        group by po.id
        having sum(ol.quantity * ol.unit_price) > :minimum""")
    List<Long> findIdsWithTotalAbove(BigDecimal minimum);    // hand-written
↳ SQL when needed
}

```

save(order) writes both the purchase_order and the order_line rows, and findById reads them together. The lines have no repository of their own, because they are part of the aggregate.

Expected output:

```

== 3.4 Spring Data JDBC aggregate
order #1 with 1 line(s), total 240.00

```

Its test: order/PurchaseOrderRepositoryTest (@DataJdbcTest)

★ Tip

Spring Data JDBC is deliberately simpler than JPA: no lazy loading, no “dirty checking” and no first-level cache. What you save is what gets written, which makes its behaviour predictable.

3.5 Transactions

Goal: make the stock decrease and the order insert one consistent operation.

```

@Transactional    // begin ... commit, or rollback on a
↳ RuntimeException
public long placeOrder(String customerEmail, Map<String, Integer> quantities) {
    audit.record("Checkout started by " + customerEmail);    // separate transaction
↳ (REQUIRES_NEW)

    Set<OrderLine> lines = new HashSet<>();
    quantities.forEach((isbn, quantity) -> {
        Book book = books.findByIsbn(isbn).orElseThrow(() -> new
↳ IllegalArgumentException("Unknown ISBN " + isbn));

```

```

        if (book.stock() < quantity) {
            throw new OutOfStockException(isbn, quantity, book.stock()); // →
            ↪ everything is rolled back
        }
        books.changeStock(isbn, -quantity);
        lines.add(new OrderLine(isbn, quantity, book.price()));
    });

    long orderId = Objects.requireNonNull(orders.save(PurchaseOrder.create(customerEmail,
        ↪ lines)).id());
    events.publishEvent(new OrderPlacedEvent(orderId, customerEmail)); // delivered
    ↪ after commit
    return orderId;
}

```

The audit entry must survive even when the order fails, so it is written in a transaction of its own:

```

@Component
public class AuditLog {

    private final JdbcClient jdbc;

    public AuditLog(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }

    @Transactional(propagation = Propagation.REQUIRES_NEW) // suspend the caller's
    ↪ transaction, commit on its own
    public void record(String message) {
        jdbc.sql("insert into audit_log (message) values (?)").param(message).update();
    }
}

```

3.6 @TransactionalEventListener

Goal: react to work **only after it has been committed**. A rolled-back order must not send the customer an e-mail.

```

@Component
public class OrderNotifier {

    private final JdbcClient jdbc;

    public OrderNotifier(JdbcClient jdbc) {

```

```

        this.jdbc = jdbc;
    }

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT) // the default
    ↪ phase
    @Transactional(propagation = Propagation.REQUIRES_NEW) // the original
    ↪ transaction is over
    public void onOrderPlaced(OrderPlacedEvent event) {
        jdbc.sql("insert into notification (message) values (?)")
            .param("Siparişiniz alındı / Order received: #" + event.orderId() + " → "
                ↪ + event.customerEmail())
            .update();
    }
}

```

Expected output (3.5 and 3.6, first run on a clean database):

```

== 3.5 + 3.6 Transactions and transactional events
checkout OK → order #2, stock of Spring in Action now 3
checkout FAILED → Only 2 of 99 copies of 9780134757599 in stock
audit_log rows: 2 (both attempts, thanks to REQUIRES_NEW)
notification rows: 1 (only after a commit)

```

Its test: checkout/CheckoutServiceTest. It checks that a successful order commits everything. For a failed order it checks that the stock and the order are rolled back while the audit entry stays.

i Note

The data lives in a Docker volume and survives restarts. For a clean start: `docker compose --profile postgres down -v`

3.7 Testing with Testcontainers

Goal: run the tests against the same database engine as production.

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    // same image as compose.yaml; tell Testcontainers it behaves like the official
    ↪ "postgres" image
    static final DockerImageName IMAGE =
        ↪ DockerImageName.parse("pgvector/pgvector:0.8.6-pg18")
        .asCompatibleSubstituteFor("postgres");
}

```

```
// static: one container per JVM, shared by every cached Spring test context
static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(IMAGE);

@Bean
@ServiceConnection
PostgreSQLContainer postgres() {
    return POSTGRES;
}
```

Test classes @Import this configuration:

Annotation	What is loaded	Transaction
@JdbcTest	DataSource, JdbcClient, Flyway	Rolled back after every test
@DataJdbcTest	+ Spring Data JDBC repositories	Rolled back after every test
@SpringBootTest	The whole application	Real commits. The test cleans up its own data

! Warning

Because the container is static, all test classes share the same database. Tests that commit (@SpringBootTest) must clean up in @BeforeEach/@AfterEach, or look only at their own rows.

4. Common Mistakes and Best Practices

! Warning

In PostgreSQL, once an SQL error happens inside a transaction, every following statement in that transaction is rejected (current transaction is aborted). If you want to try several constraint violations, put each one into its own test.

- **Do:** always pass SQL parameters as :name or ?. Never build SQL by concatenating strings (SQL injection).
- **Don't:** call a @Transactional method from inside the same class. The proxy is bypassed (self-invocation).
- **Do:** protect business rules (such as stock $\geq$ 0) both in code and as a check constraint in the database.
- **Don't:** assume checked exceptions roll back. Use rollbackFor if you need that.

- **Do:** use `@Transactional(readOnly = true)` on service methods that only read.
- **Don't:** expect H2 to “behave like PostgreSQL” in tests. Use the real database with Testcontainers.

5. Summary

- Docker Compose support starts the development database, Testcontainers the test database, automatically.
- Flyway manages the schema with versioned SQL files. Applied migrations are never changed.
- JdbcClient runs SQL through a lean API and maps the results to records.
- Spring Data JDBC stores aggregates as a whole. Spring generates the repository interfaces.
- `@Transactional` gives an all-or-nothing guarantee. `REQUIRES_NEW` opens an independent transaction.
- `@TransactionalEventListener(AFTER_COMMIT)` reacts only to successful commits.

Next step: [Exercises](#)

6. Further Reading

- [Spring Framework — JDBC Core \(JdbcClient\)](#)
- [Spring Framework — Transaction Propagation](#)
- [Spring Framework — Transaction-bound Events](#)
- [Spring Data JDBC](#)
- [Spring Boot — SQL Databases · Database Initialization \(Flyway\)](#)
- [Spring Boot — Testcontainers · Docker Compose Support](#)
- [Flyway Documentation](#)